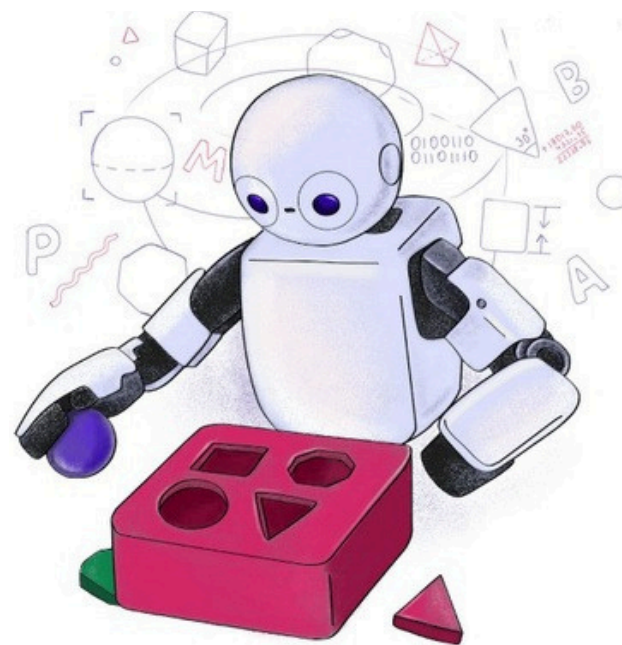


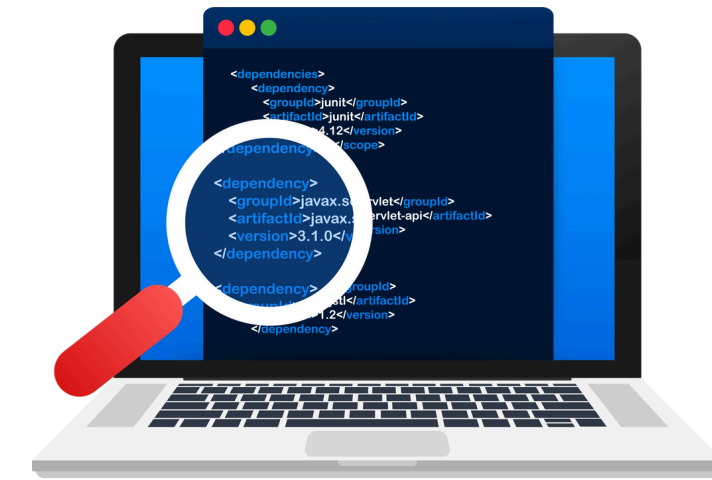
TP558 - Tópicos avançados em
Machine Learning:
**Learning to Compare: Relation
Network for Few-Shot Learning.**



Inatel

Ana Cecília Silveira Fernandes
ana.fernandes@mtel.inatel.br

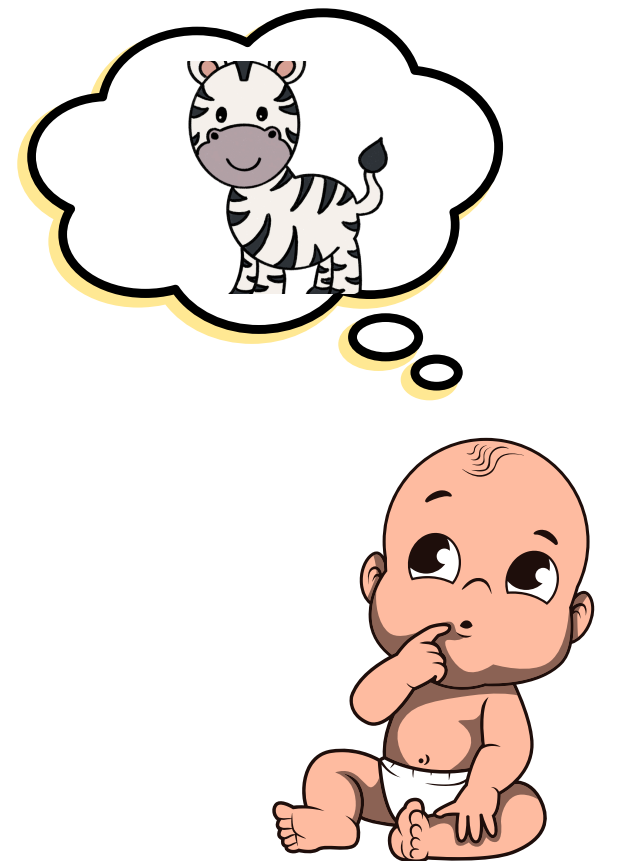
Introdução

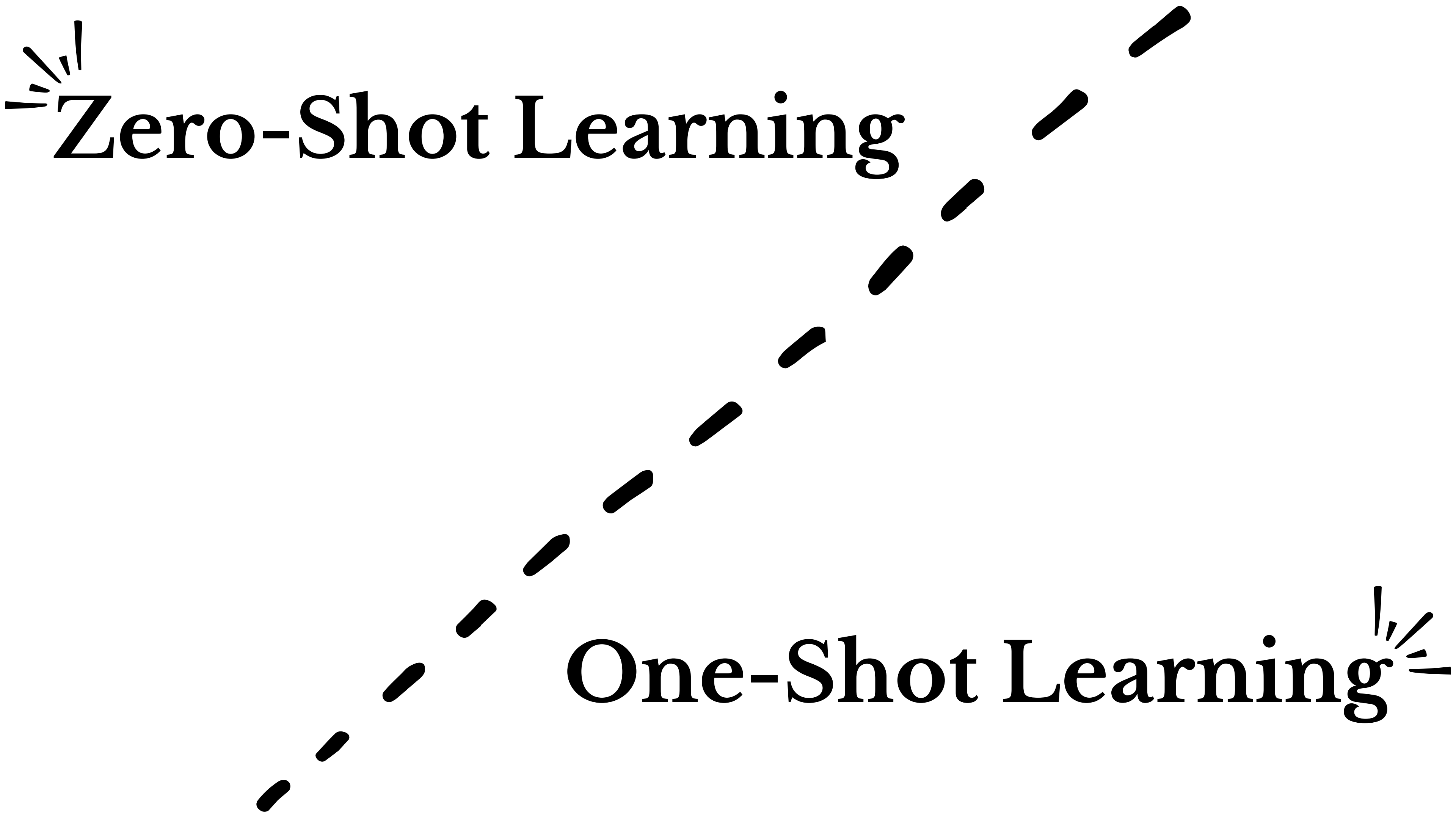


- Em aprendizado de máquina, sabemos que bons modelos geralmente precisam de grandes quantidades de dados rotulados e muito poder computacional.
- Muitos problemas práticos envolvem conjuntos de dados pequenos. O aprendizado profundo irá sobreajustar (overfit) esses conjuntos de dados e não irá generalizar.



- Humanos se saem bem em tarefas de classificação usando apenas alguns exemplos: uma criança pode selecionar imagens de zebras tendo apenas um único exemplo.



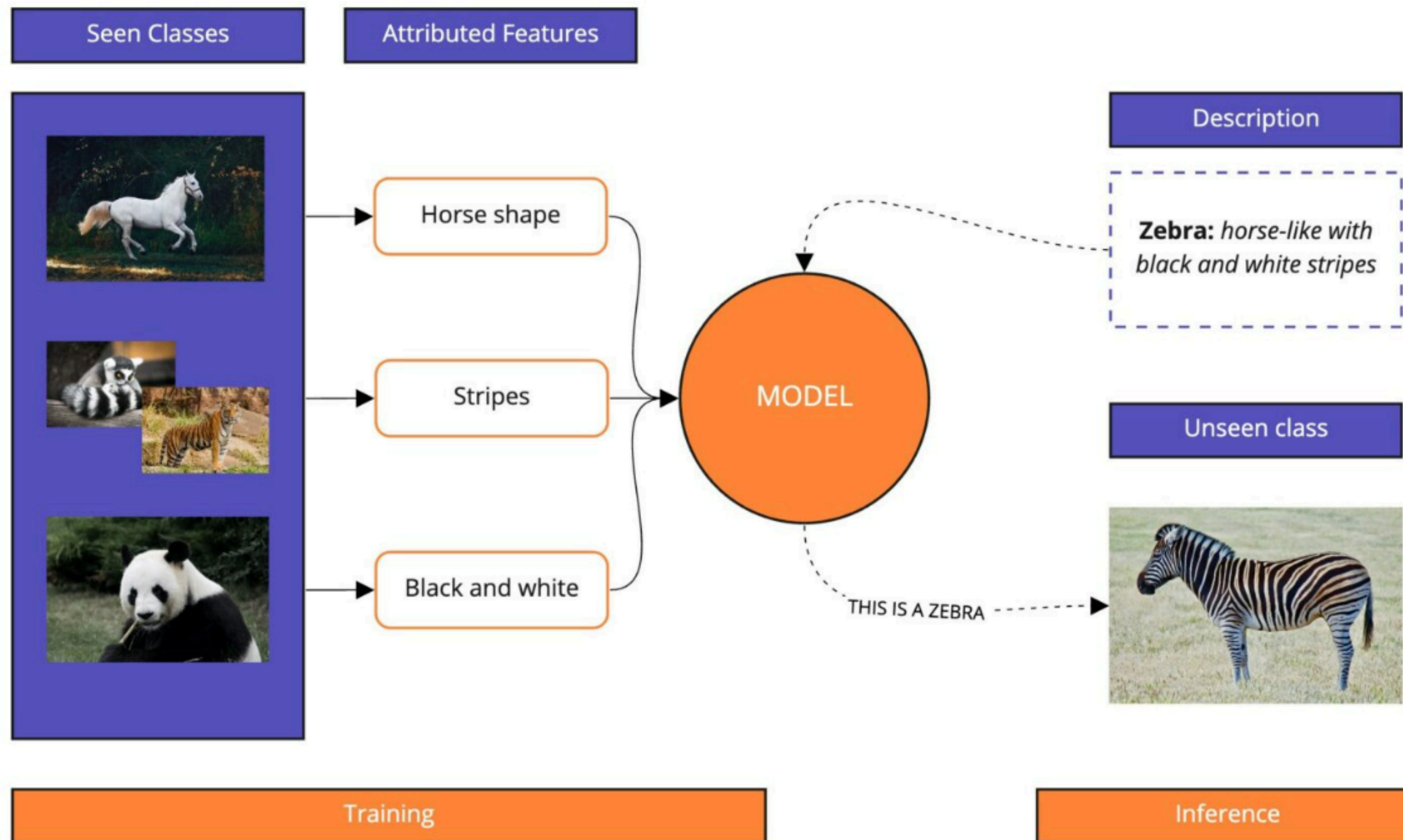


Zero-Shot Learning

One-Shot Learning

Zero-Shot Learning (ZSL)

- Modelo capaz de reconhecer classes nunca vistas no treinamento
- Utiliza descrições semânticas (ex.: atributos, embeddings de palavras, textos) para generalizar.
- Dispensa exemplos rotulados das novas classes.
- Permite classificar tarefas ou categorias inéditas com base em conhecimento prévio.
- Exemplo: reconhecer uma zebra sem nunca ter visto imagens dela, apenas pela descrição “animal parecido com cavalo, mas com listras”.
- Aplicações:
 - NLP → classificação de textos em categorias não vistas, tradução entre idiomas não treinados diretamente.
 - Visão computacional → identificação de objetos inéditos
 - Sistemas de recomendação → sugerir itens novos apenas pela descrição.



Zero-Shot Learning (ZSL)

Treino

Modelo
aprende
classes vistas
+ atributos.



Teste

Aparece
uma classe
não vista
(zebra).

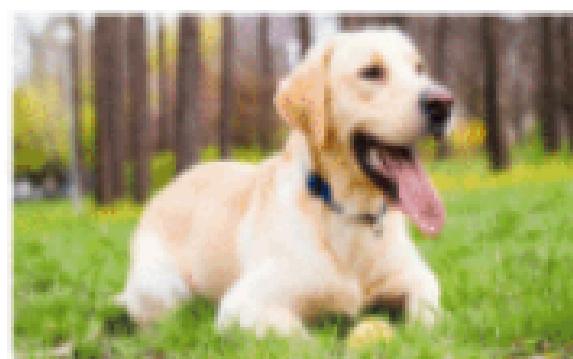
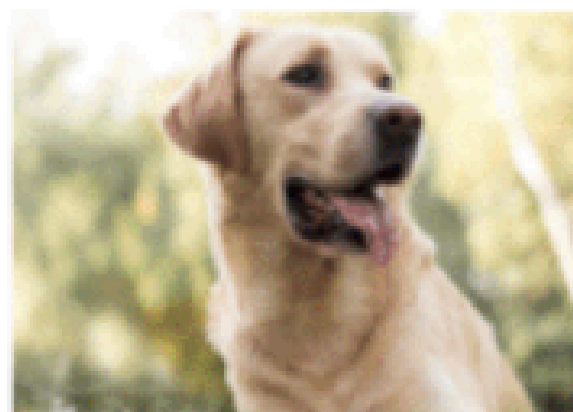


Resultado

Modelo usa
atributos para
inferir que é
uma zebra.

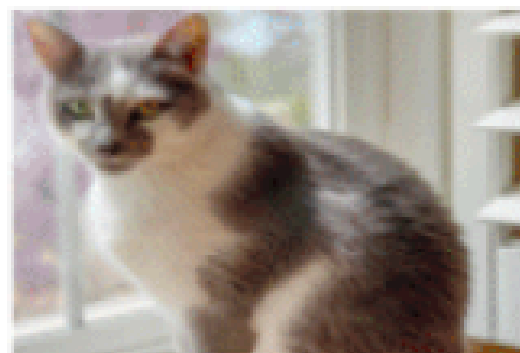
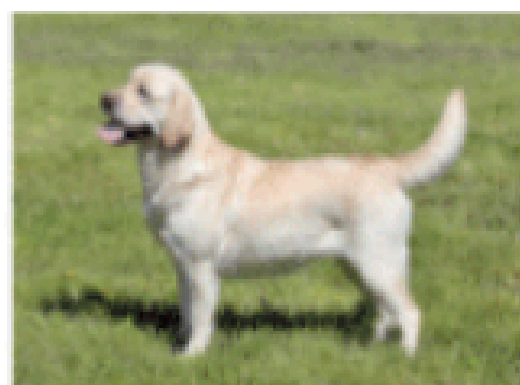
One-Shot Learning (OSL)

- Modelo aprende a reconhecer uma nova classe com apenas 1 exemplo
- Muito usado em tarefas de comparação e verificação (ex.: redes siamesas)
- Foca em aprender boas representações que permitam medir similaridade
- Exemplo: identificar uma nova pessoa com apenas uma foto de referência
- Diferença para Zero-Shot: OSL precisa de 1 amostra da nova classe, enquanto ZSL não precisa de nenhuma
- Aplicações:
 - Reconhecimento facial e biometria
 - Deduplicação de imagens em e-commerce
 - Assinaturas e verificação de identidade



One shot
learning

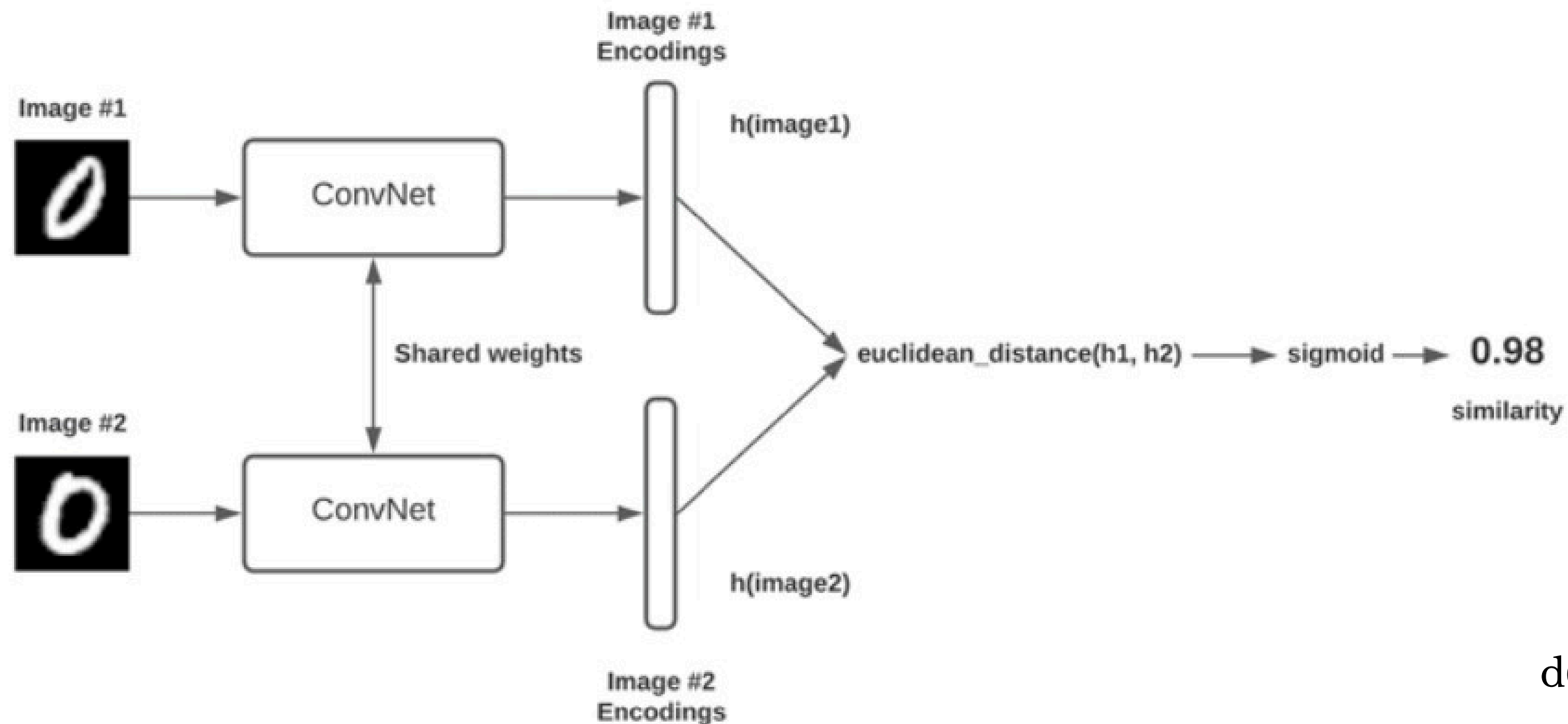
Same



One shot
learning

Different

Rede Neural Siamesa (SNN)



$$d(x, y) = \sqrt{\sum (x_i - y_i)^2}$$

Pequena → Semelhantes

Grande → Diferentes

Introdução à Relation Network



- Em 2018, Sung e colaboradores propuseram a **Relation Network**. Essa abordagem leva a ideia das siamesas um passo além, permitindo que a própria rede aprenda a função de comparação de forma end-to-end, resultando em maior flexibilidade e desempenho em cenários de few-shot learning.



GitHub

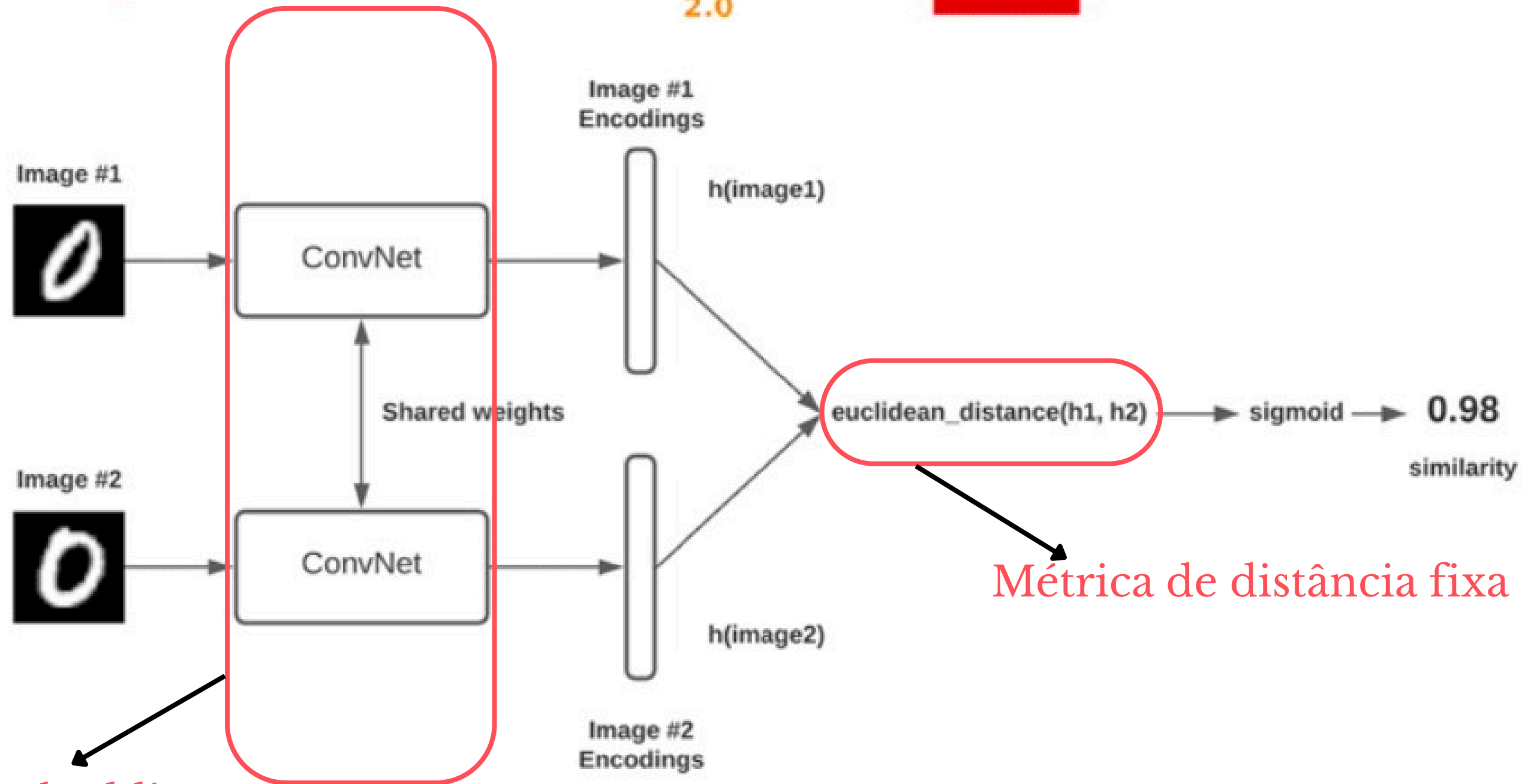


Fundamentação Teórica



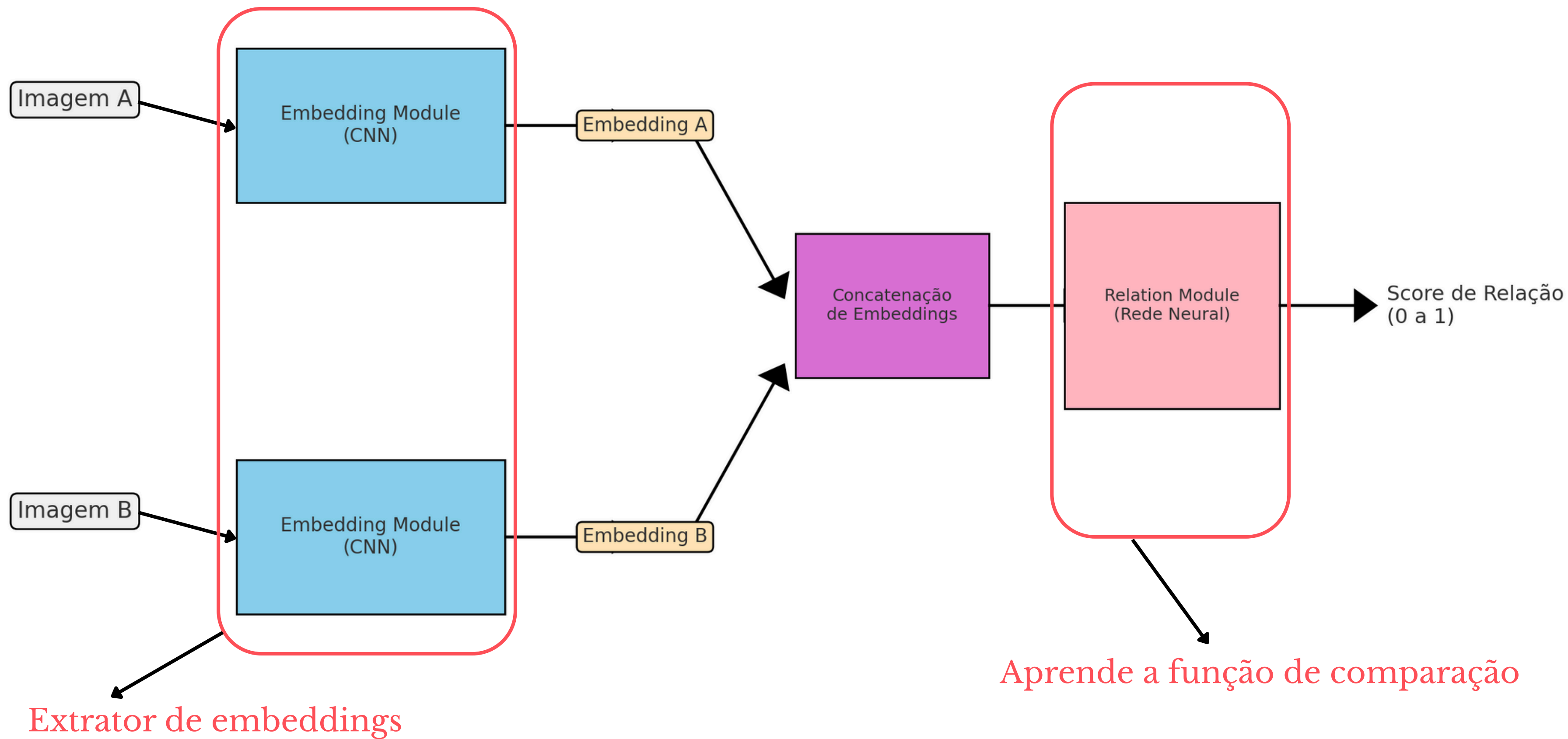
End-to-End?

- Significa que todo o processo, desde extrair as características até aprender a função de comparação, é treinado junto em um único modelo, de ponta a ponta.



Extrator de embeddings

Métrica de distância fixa



Trabalhos Relacionados

- Fine-tune: uma rede existente, pré-treinada em um conjunto de dados relacionado, é ajustada (fine-tuned) com o conjunto de dados de treinamento alvo.
- Redes Neurais Recorrentes com Memória: utilizam memória interna para armazenar informações de exemplos anteriores e realizar classificações.
- Aprendizado de Métrica: aprende-se um conjunto de projeções e, em seguida, aplica-se um classificador linear para tomar a decisão final.
- Redes Siamesas: treinam-se redes para produzir embeddings que depois podem ser usados com um classificador linear.

- A Relation Network (RN) possui vantagens em relação às abordagens citadas anteriormente. Ela realiza **meta-learning** em um grande conjunto de dados relacionado, aprendendo simultaneamente tanto o embedding quanto uma métrica não linear. Esse treinamento de ponta a ponta torna o processo mais simples e mais poderoso do que seus predecessores.

O meta-learning é como dar ao modelo uma caixa de ferramentas para aprender mais rápido em cenários de few-shot ou zero-shot learning, onde os dados disponíveis são muito limitados.

Exemplo de um método simples para classificação usando apenas dados de treinamento limitados: **Busca do Vizinho Mais Próximo.**

- É um método não-paramétrico e super simples de classificação.
- Funciona assim:
 - Você tem um conjunto de suporte (support set) com alguns exemplos rotulados de cada classe.
 - Para classificar uma nova imagem (query), você mede a distância dela para cada exemplo do suporte.
 - Você atribui a classe do exemplo mais próximo (menor distância).

Support Set



Dog



Cat

**Euclidean
distance**

Query



Cat

Nearest Neighbor Search

Vantagens

- Simples

- Rápido

Desvantagens

- Falsos vizinhos

- A métrica de distância é simples

Relation Network (RN)

- Relation Network: aprende um módulo de relação (uma pequena rede neural) que aprende como comparar, capturar relações muito mais complexas do que só distância Euclidiana.

Metodologia RN

Três conjuntos de dados diferentes:

- Treinamento (training set): usado para aprender.
- Suporte (support set): contém poucos exemplos por classe (o “manual” de referência).
- Teste (testing set): contém novas imagens a serem classificadas.



Diferença para aprendizado supervisionado comum: no RN, as classes do treinamento são diferentes das classes de suporte/teste, forçando o modelo a aprender a comparar, e não decorar.

- O treinamento acontece em episódios, cada episódio simula uma tarefa de few-shot.
- Em cada episódio, o modelo pega algumas classes (C-way) e alguns exemplos por classe (K-shot) para formar:
 - Sample set (equivalente ao support set): imagens conhecidas, com rótulo.
 - Query set: imagens a serem classificadas comparando com o sample.

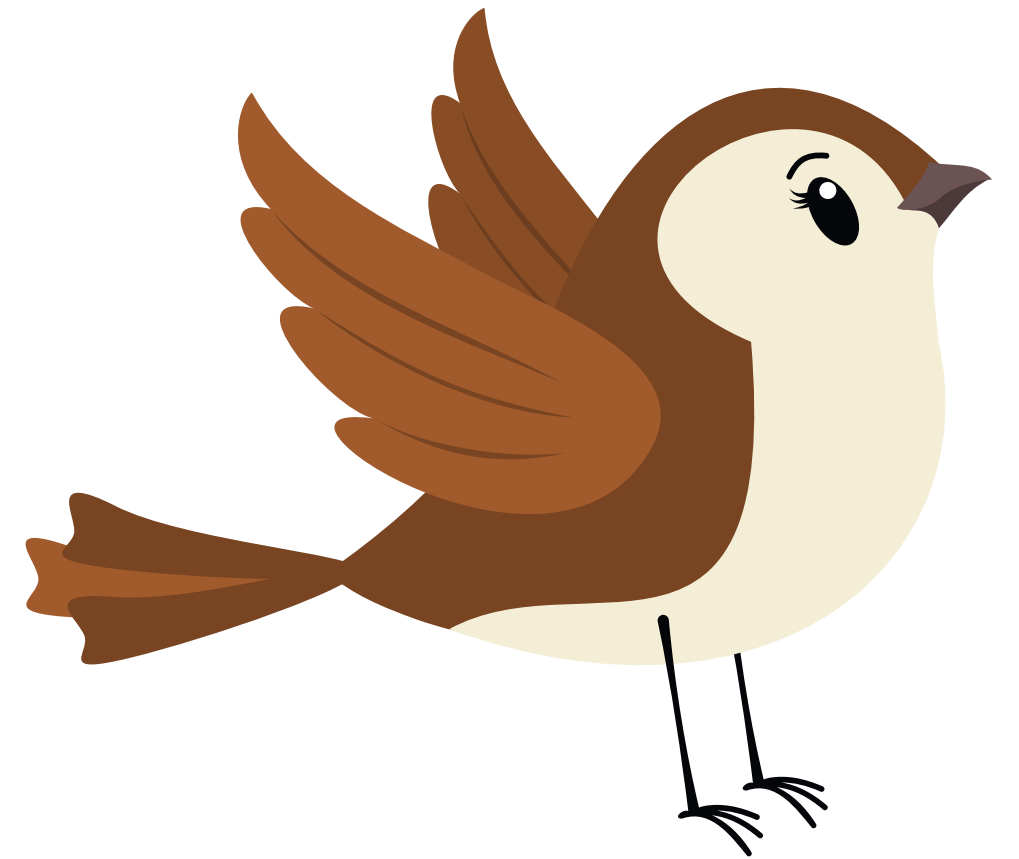
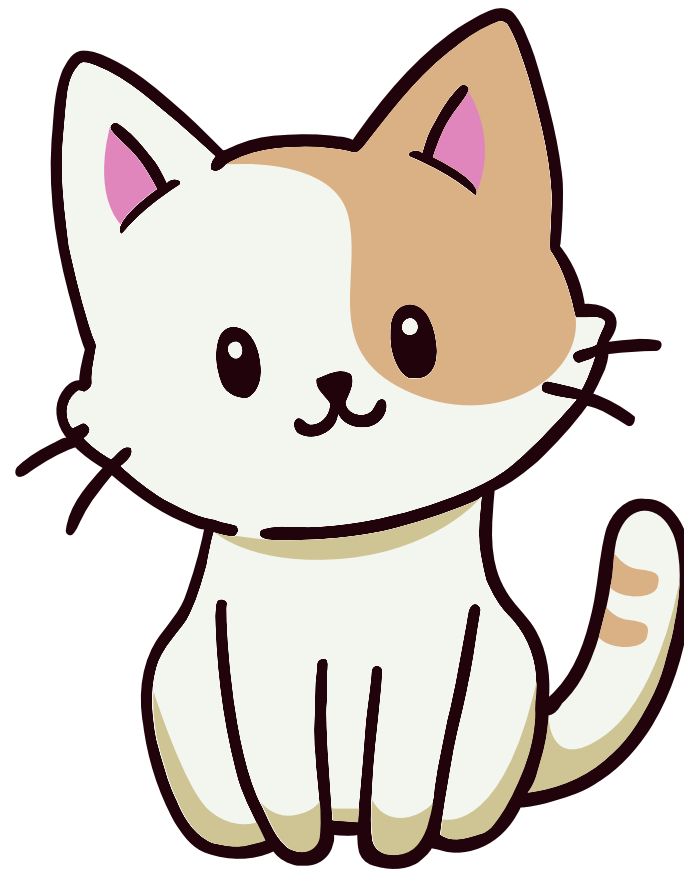
Estrutura de um episódio

3° Passo: Selecionar
imagens adicionais
da mesma classe

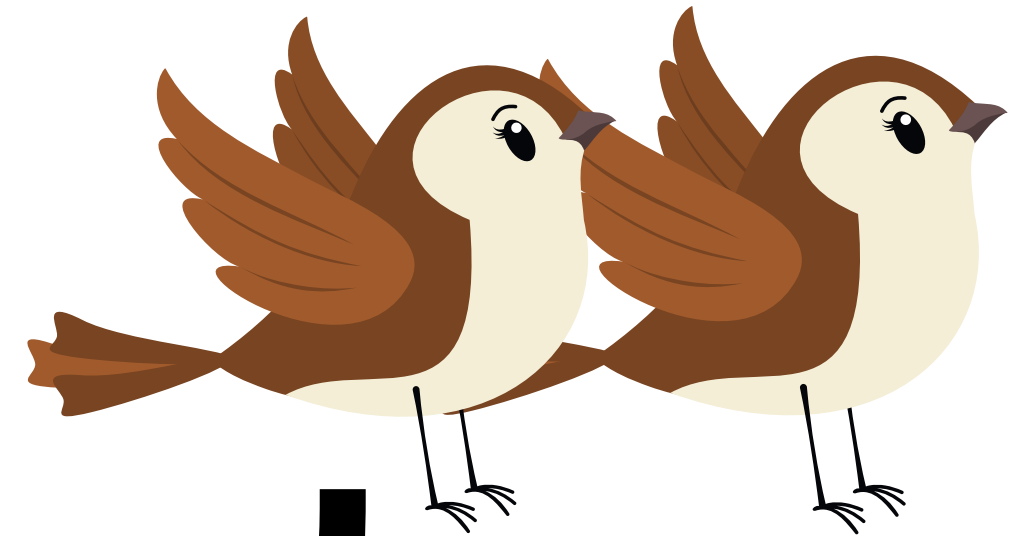
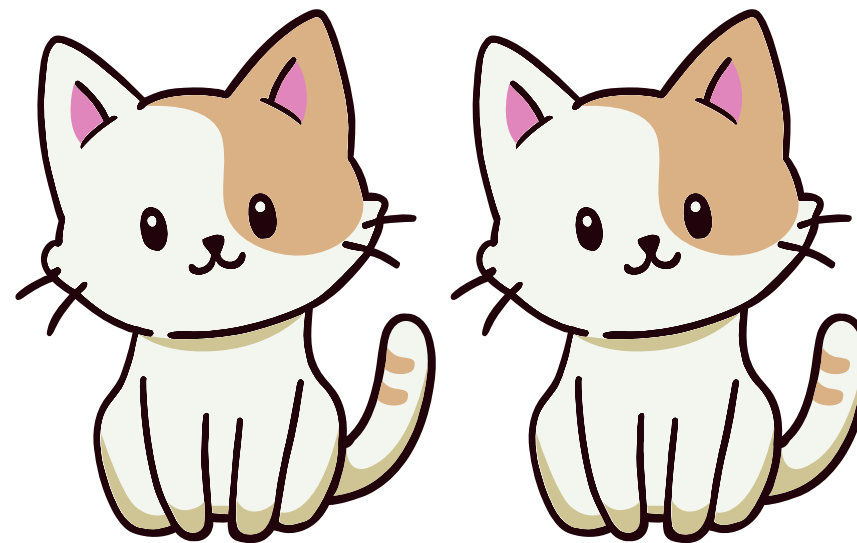
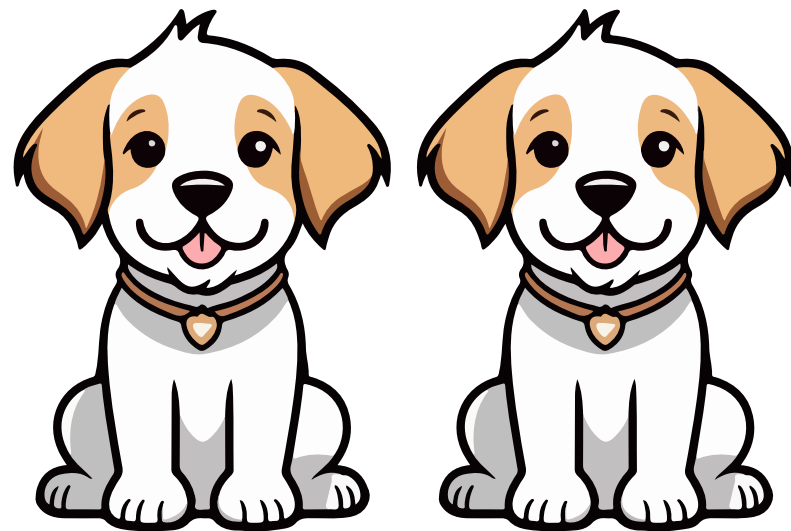
2° Passo: Escolher
K exemplos por
classe (K-shot)

1° Passo: Escolher
C classes (C-way)

1º Passo: Escolher C classes (C-way)

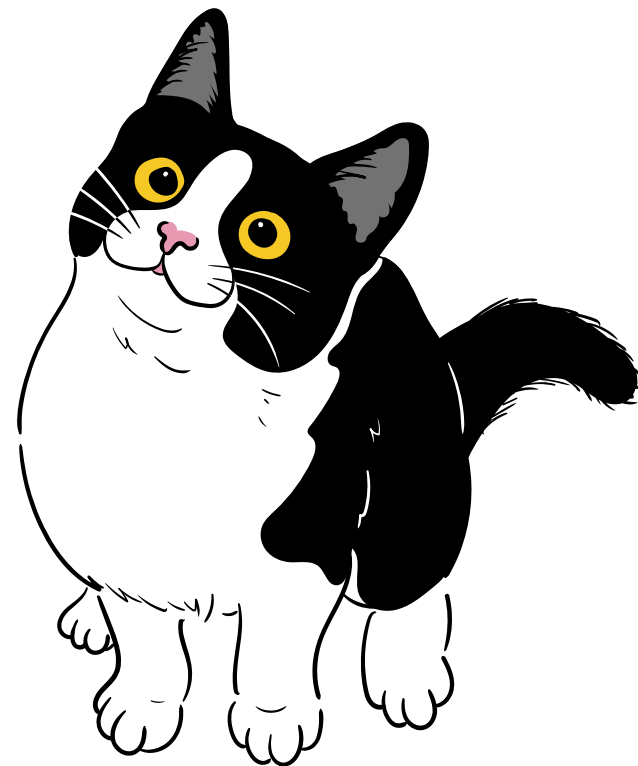


2º Passo: Escolher
K exemplos por
classe (K-shot):



Sample set

3° Passo: Selecionar
imagens adicionais
da mesma classe



Query set

$$Query = \{(x_i, y_i)\}_{i=1}^{n_q}$$

$$Sample = \{(x_i, y_i)\}_{i=1}^{n_s \times C}$$

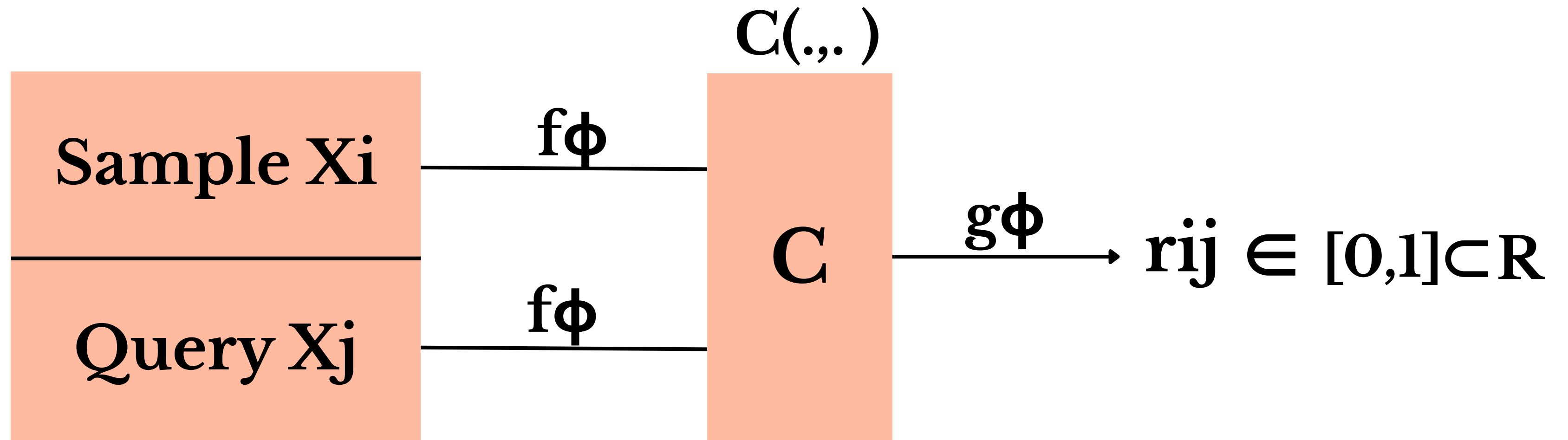
Diferença essencial

- Sample set (suporte): fornece exemplos rotulados (a “base” para comparar).
- Query set: fornece exemplos novos da mesma classe, que a rede precisa classificar comparando com o suporte.

Processo de Comparação no RN

- Tanto os vetores do sample quanto do query são embutidos (embedded) usando o módulo de embedding $f\phi$.
- Depois do embedding, os dois vetores são concatenados.
- O módulo de relação $g\phi$ então mapeia o vetor concatenado para um número entre 0 e 1, onde 1 indica que são muito similares e 0 indica nenhuma similaridade.

Processo de Comparação no RN



Função de perda - Mean Squared Error

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

Outras funções de perda aplicáveis em RN

$$L = -\frac{1}{N} \sum_i [y_i \log(r_i) + (1 - y_i) \log(1 - r_i)]$$

- **Binary Cross-Entropy (BCE)**
 - Muito usada em problemas binários (mesma classe vs diferente).
 - Interpreta o relation score como uma probabilidade.
 - Mais comum hoje em tarefas de few-shot porque lida melhor com outputs sigmóides.

Outras funções de perda aplicáveis em RN

$$L = y \cdot D^2 + (1 - y) \cdot \max(0, m - D)^2$$

- **Contrastive Loss (inspirada em SNNs)**
 - Onde D é a distância entre embeddings, e m é uma margem definida.
 - Se são da mesma classe $\rightarrow$ minimizar a distância.
 - Se são diferentes $\rightarrow$ distância precisa ser maior que a margem m .

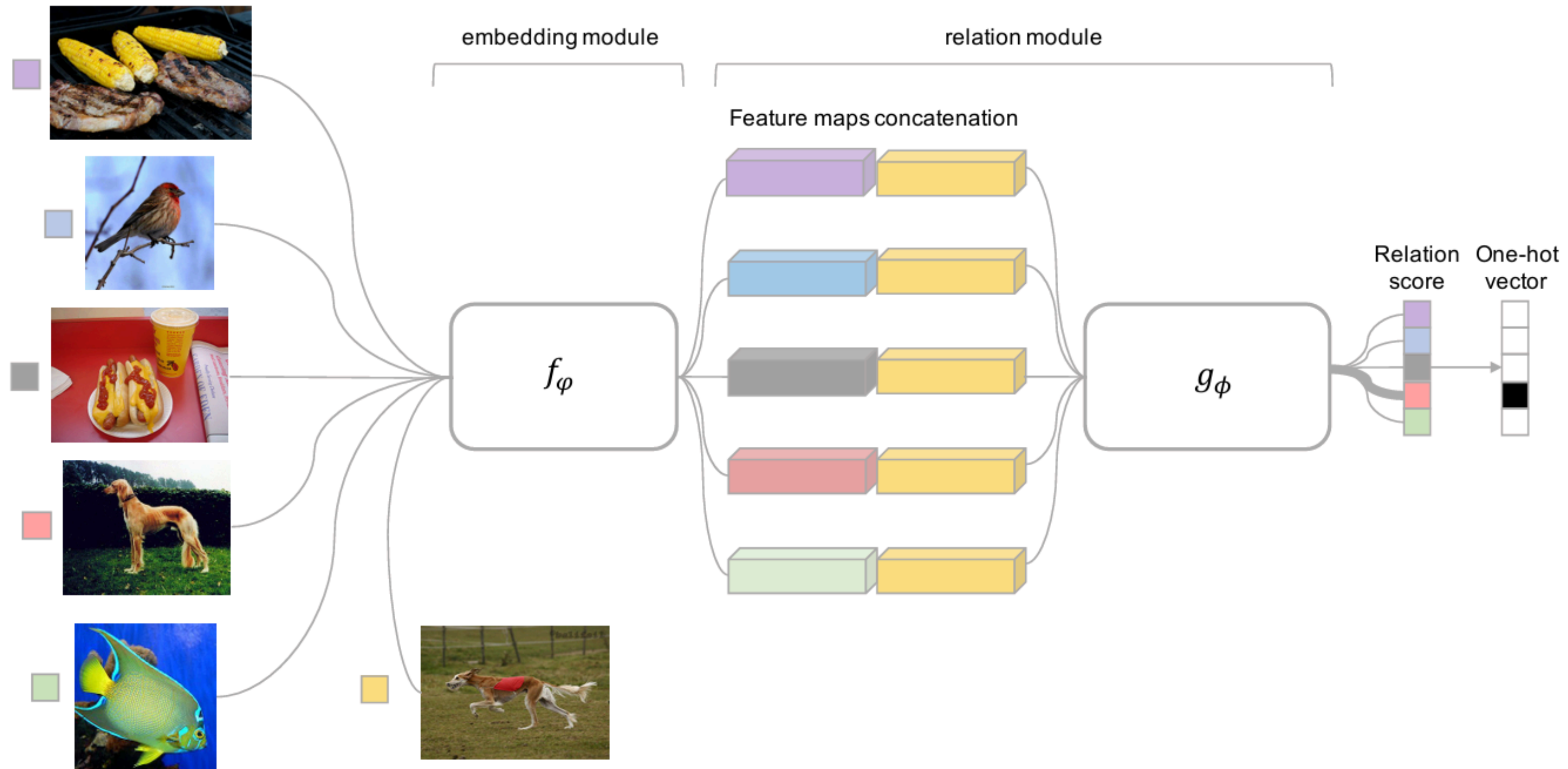
Outras funções de perda aplicáveis em RN

$$L = \max(0, D(a, p) - D(a, n) + m)$$

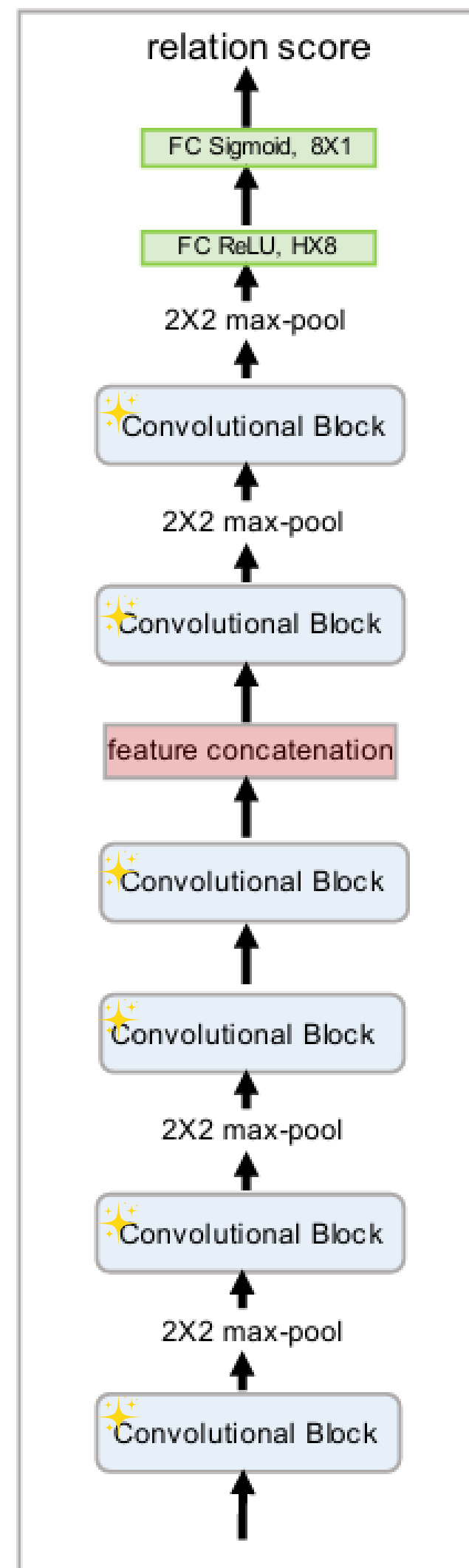
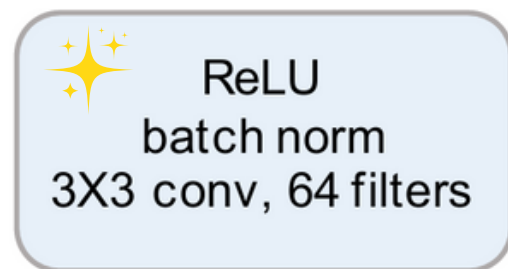
Triplet Loss:

- Usa três exemplos:
- Anchor (a), Positivo (p) da mesma classe, e Negativo (n) de outra classe.
- Objetivo: garantir que o positivo esteja mais próximo do que o negativo, por pelo menos uma margem m.
- Muito popular em verificação de identidade (faces, assinaturas, etc.).

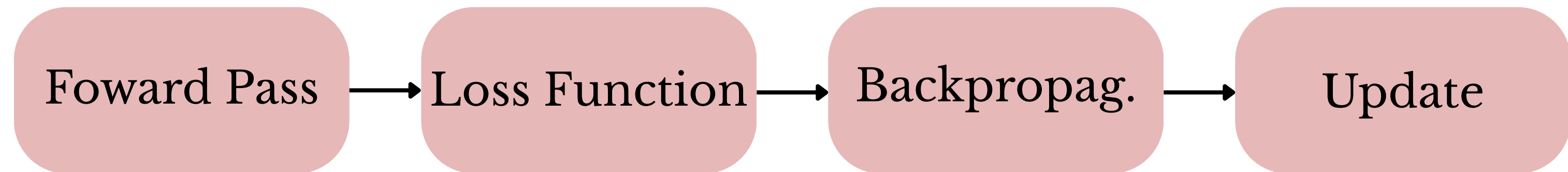
5-way 1-shot



Arquitetura da RN



Otimização



Foward Pass

Query + sample passam pelo Embedding Module
→ Relation Module → Gera relation score (0 a 1).

Loss Function

Eles usam mean square error (MSE loss) entre o relation score e o rótulo da classe (0 ou 1).

Backpropag.

O gradiente da loss é propagado tanto para o relation module quanto para o embedding module.

Update

- Otimizador Adam (às vezes SGD em experimentos comparativos).
- Learning rate ajustado durante os episódios de treino.

Por que SGD ou Adam?

SGD

- Atualiza os pesos com base em pequenos lotes (batches) de dados.
- Simples, robusto e funciona bem, mas pode ser lento em convergência, especialmente em problemas com muitos parâmetros.

Adam

- Mantém médias móveis do gradiente (1º momento) e do quadrado do gradiente (2º momento).
- Isso ajusta automaticamente a taxa de aprendizado para cada parâmetro.

Vantagens e desvantagens

Vantagens	Desvantagens
• Não precisa de fine-tuning	• Custo computacional
• Aprende a métrica automaticamente	• Escalabilidade limitada
• Treinamento end-to-end	• Dependência do dataset de treino
• Alto desempenho em few-shot	• Menor interpretabilidade

Exemplo(s) de aplicação

Dataset Omniglot

- Contém 1623 classes (caracteres) de 50 alfabetos diferentes.
- Cada classe tem 20 exemplos.
- 1200 caracteres para treinamento (Sample/Query)
- 423 caracteres para teste (Support/Test)

Experimentos:

- 5-way 1-shot
- 5-way 5-shot
- 20-way 1-shot
- 20-way 5-shot

Model	Fine Tune	5-way Acc.		20-way Acc.	
		1-shot	5-shot	1-shot	5-shot
MANN [32]	N	82.8%	94.9%	-	-
CONVOLUTIONAL SIAMESE NETS [20]	N	96.7%	98.4%	88.0%	96.5%
CONVOLUTIONAL SIAMESE NETS [20]	Y	97.3%	98.4%	88.1%	97.0%
MATCHING NETS [39]	N	98.1%	98.9%	93.8%	98.5%
MATCHING NETS [39]	Y	97.9%	98.7%	93.5%	98.7%
SIAMESE NETS WITH MEMORY [18]	N	98.4%	99.6%	95.0%	98.6%
NEURAL STATISTICIAN [8]	N	98.1%	99.5%	93.2%	98.1%
META NETS [27]	N	99.0%	-	97.0%	-
PROTOTYPICAL NETS [36]	N	98.8%	99.7%	96.0%	98.9%
MAML [10]	Y	98.7 $\pm$ 0.4%	99.9 $\pm$ 0.1%	95.8 $\pm$ 0.3%	98.9 $\pm$ 0.2%
RELATION NET	N	99.6 $\pm$ 0.2%	99.8 $\pm$ 0.1%	97.6 $\pm$ 0.2%	99.1 $\pm$ 0.1%

Implementação



```
[Ep  10] loss=0.2262  acc=40.0%
[Ep  20] loss=0.1966  acc=66.7%
[Ep  30] loss=0.1566  acc=86.7%
[Ep  40] loss=0.1484  acc=73.3%
[Ep  50] loss=0.1562  acc=86.7%
[Ep  60] loss=0.1273  acc=86.7%
[Ep  70] loss=0.1597  acc=60.0%
[Ep  80] loss=0.1371  acc=73.3%
[Ep  90] loss=0.0990  acc=86.7%
[Ep 100] loss=0.1127  acc=86.7%
Test few-shot accuracy over 20 episodes: 84.00%
```

- Aqui, a rede foi avaliada em 20 episódios de teste usando classes nunca vistas no treino.
- Ela atingiu 84% de acurácia média.
- Isso significa que, mesmo sem nunca ter visto essas classes, ela conseguiu classificar 84% das queries corretamente com base em poucos exemplos de suporte (few-shot).



Comparação com outros algoritmos

	Siamese Nets	Matching Nets	Prototypical Nets	Relation Nets
Tipo de saída	Score de similaridade (0/1, mesma ou diferente classe).	Probabilidade via atenção sobre o suporte.	Vetor protótipo por classe + distância ao query.	Relation score $\in [0,1]$ aprendido por rede neural.
Quando funciona melhor	Problemas simples de verificação (assinatura, face).	Poucos exemplos com suporte variado.	Classes bem representadas por centróides.	Cenários complexos, relações não lineares entre classes.
Reutilização	Fácil reutilizar embeddings em outros datasets.	Reuso limitado (depende de atenção aprendida).	Protótipos podem ser recalculados facilmente.	Necessita rodar a rede para cada par query–sample.
Complexidade de treino	Baixa (CNN + métrica fixa).	Média (atenção + embeddings).	Média (média dos embeddings + distância).	Maior (CNN $f\phi$ + CNN/MLP $g\phi$, comparações par-a-par).
Uso prático	Verificação 1:1 (mesma/diferente).	Classificação few-shot multi-classe.	Classificação few-shot multi-classe.	Classificação few-shot multi-classe com métrica aprendida.
Vantagem principal	Simplicidade, baixa necessidade de dados.	Atenção melhora generalização.	Protótipos reduzem custo computacional.	Métrica aprendida end-to-end, expressiva.
Desvantagem	Métrica fixa pode falhar em relações complexas.	Pode sobreajustar a distribuição do suporte.	Assume que classes são bem representadas por médias.	Custo maior (precisa comparar query com todos os samples).

Conclusão

- Aprendem similaridade end-to-end
- Superam métodos clássicos (Siamese, Matching, Prototypical)
- Generalizam bem para classes nunca vistas
- Simples no conceito, eficientes em few-shot
- Limitação: alto custo computacional em muitos exemplos

Perguntas?

Referências

<https://arxiv.org/pdf/1711.06025>

<https://github.com/floodsung>

<https://pyimagesearch.com/2020/11/30/siamese-networks-with-keras-tensorflow-and-deep-learning/>

<https://www.cs.cmu.edu/~rsalakhu/papers/oneshot1.pdf>

Obrigada!

Formulário

